

# Codificação Para Front-end

13

---

2026

SENAI - CRTI

# Layout Moderno I

## Flexbox

Modernizando o layout do nosso e-commerce com a técnica mais poderosa do CSS moderno — tornando o código mais limpo, flexível e profissional.

DESENVOLVIMENTO FRONT-END



Situação de Aprendizagem

# O Desafio do Nosso E-Commerce

O layout atual do nosso e-commerce foi construído com técnicas mais antigas: `inline-block` e posicionamento manual. Embora funcional, esse código é difícil de manter, pouco flexível e não está preparado para se adaptar a diferentes tamanhos de tela.

Nesta aula, sua missão é **modernizar esse layout utilizando Flexbox** — uma abordagem nativa do CSS que resolve de forma elegante os principais desafios de alinhamento e distribuição de elementos.

## Capacidades a desenvolver

- Utilizar Flexbox para construção de layouts
- Compreender eixo principal e eixo cruzado
- Alinhar e distribuir elementos corretamente
- Substituir técnicas antigas por soluções modernas

# As Limitações do Layout Anterior

inline-block

Gera espaços brancos indesejados entre os elementos, exigindo hacks no HTML ou fontes zeradas para corrigir.

Posicionamento Manual

Usar `float` ou `position` para layout é frágil, difícil de manter e quebra com frequência em telas menores.

Falta de Flexibilidade

Centralizar verticalmente um elemento era um desafio enorme. Distribuir espaço entre itens exigia cálculos manuais.

Responsividade Difícil

Adaptar layouts baseados em `inline-block` para mobile requer reescrever grandes partes do CSS.



**CSS FLEXBOX**

# O que é Flexbox?

**Flexbox** (Flexible Box Layout) é um modelo de layout CSS projetado para distribuir espaço e alinhar elementos dentro de um container de forma dinâmica e inteligente — mesmo quando o tamanho desses elementos é desconhecido ou muda dinamicamente.

Com uma única propriedade no elemento pai, todos os filhos diretos passam a se comportar de forma coordenada, respondendo ao espaço disponível sem a necessidade de cálculos manuais.

- ❗ Flexbox opera em **uma dimensão** por vez: linha ou coluna. Para layouts bidimensionais complexos, usaremos CSS Grid na próxima aula.

```
/* Antes: técnica antiga */
.menu li {
  display: inline-block;
  margin-right: -4px; /* hack! */
}

/* Depois: Flexbox moderno */
.menu {
  display: flex;
  gap: 16px;
}
```

# Flex Container vs. Flex Items

O Flexbox funciona em dois níveis hierárquicos distintos. Compreender essa relação é a base para dominar toda a tecnologia.

## Flex Container

É o elemento **pai** que recebe a propriedade `display: flex`. Ele define o contexto flex e controla como os filhos serão distribuídos. As propriedades aplicadas aqui afetam todos os filhos diretos.

Propriedades do container: `justify-content`, `align-items`, `flex-wrap`, `flex-direction`, `gap`



## Flex Items

São os **filhos diretos** do container flex. Eles se organizam automaticamente dentro do container seguindo as regras definidas. Cada item pode ter propriedades individuais para personalizar seu comportamento.

Propriedades dos items: `flex-grow`, `flex-shrink`, `flex-basis`, `order`, `align-self`

```
/* O .menu é o CONTAINER – os <li> são os ITEMS */
.menu {
  display: flex; /* ativa o contexto flex */
  gap: 24px;
}
.menu li { /* cada item pode ter estilo próprio */ }
```

# Os Dois Eixos do Flexbox

Todo container flex opera em dois eixos perpendiculares. Entender esses eixos é essencial para usar corretamente as propriedades de alinhamento e distribuição.

Eixo Principal (Main Axis)

Direção na qual os flex items são dispostos. Por padrão, é **horizontal** (da esquerda para a direita). Controlado por `flex-direction`.

Eixo Cruzado (Cross Axis)

Perpendicular ao eixo principal. Se o main axis é horizontal, o cross axis é **vertical**. Usado por `align-items` e `align-self`.

```
/* flex-direction define o eixo principal */

.container {
  display: flex;
  flex-direction: row;    /* → horizontal (padrão) */
  /* flex-direction: column; ↓ vertical */
}

/* Eixo Principal → justify-content
   Eixo Cruzado  ↓ align-items    */

.container {
  display: flex;
  justify-content: center; /* no eixo principal */
  align-items: center;    /* no eixo cruzado */
}
```

❏ Dica: quando `flex-direction: column`, os eixos se invertem! O `justify-content` passa a controlar o eixo vertical.

# justify-content — Distribuição no Eixo Principal

Esta propriedade define como os flex items são distribuídos ao longo do **eixo principal** quando há espaço sobrando no container.

flex-start

Itens agrupados no início (padrão). Útil para menus e listas alinhadas à esquerda.

center

Itens centralizados. Perfeito para centralizar conteúdo em banners e seções hero.

space-between

Primeiro e último item nas bordas, espaço igual entre os demais. Ideal para navegação.

space-around

Espaço igual ao redor de cada item. Ótimo para grades de produtos.

flex-end

Itens agrupados no final. Útil para alinhar botões e ações à direita.

```
.navbar { display: flex; justify-content: space-between; } /* logo + menu + carrinho */  
.product-grid { display: flex; justify-content: center; flex-wrap: wrap; }
```

# align-items — Alinhamento no Eixo Cruzado

Enquanto `justify-content` distribui no eixo principal, `align-items` controla o alinhamento no **eixo cruzado** (geralmente vertical). Esta é a propriedade que finalmente resolve o famoso problema de centralização vertical no CSS!

`stretch`

Itens se esticam para preencher o container (padrão).

`center`

Centralizado no eixo cruzado. Resolve centralização vertical!

`flex-start` / `flex-end`

Alinha no início ou no final do eixo cruzado.

`baseline`

Alinha pela linha de base do texto dos itens.

```
/* Centralizar vertical e horizontal
   – o sonho de todo dev CSS! */
.hero-banner {
  display: flex;
  justify-content: center; /* eixo X */
  align-items: center;     /* eixo Y */
  min-height: 400px;
}

/* Navbar: logo e menu alinhados */
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center; /* ✓ verticalmente centrado */
  padding: 0 24px;
  height: 64px;
}
```

👉 Com apenas 2 propriedades, o que antes exigia `line-height`, `vertical-align` e margin tricks agora é resolvido de forma limpa!

# flex-wrap — Quebra de Linha Automática

Por padrão, todos os flex items tentam caber em uma única linha, comprimindo-se se necessário. A propriedade `flex-wrap` muda esse comportamento, permitindo que os itens quebrem para novas linhas quando não houver espaço suficiente.



nowrap

**Padrão.** Todos os itens ficam em uma linha. Se necessário, são comprimidos ou estouram o container.



wrap

Itens quebram para novas linhas quando o espaço acaba. Essencial para grids de produtos responsivos.



wrap-reverse

Igual ao `wrap`, mas as linhas extras aparecem acima, em ordem inversa.

```
/* Grade de produtos que se adapta ao espaço disponível */
.product-grid {
  display: flex;
  flex-wrap: wrap; /* permite quebra de linha */
  gap: 24px; /* espaço uniforme entre produtos */
  justify-content: center; /* centraliza quando sobra espaço */
}
.product-card {
  flex-basis: 280px; /* largura mínima ideal de cada card */
  flex-grow: 1; /* pode crescer para preencher o espaço */
  max-width: 320px; /* mas não cresce demais */
}
```

# flex-grow e flex-shrink

Controlando o tamanho dos itens

Essas duas propriedades definem como cada flex item reage ao espaço disponível no container — crescendo para preencher o espaço ou encolhendo para caber.

flex-grow

Define o quanto um item pode **crescer** proporcionalmente em relação aos outros. `0` = não cresce (padrão). `1` = cresce para ocupar o espaço livre.

flex-shrink

Define o quanto um item pode **encolher** quando o espaço é insuficiente. `1` = encolhe (padrão). `0` = não encolhe nunca.

flex-basis

Tamanho inicial do item antes de crescer ou encolher. Funciona como `width`, mas dentro do contexto flex.

```
/* Atalho: flex: grow shrink basis */

/* Sidebar + conteúdo principal */
.sidebar {
  flex: 0 0 260px; /* fixo: não cresce, não encolhe */
}
.main-content {
  flex: 1 1 auto; /* flexível: cresce e encolhe */
}

/* Todos os cards com mesmo tamanho */
.product-card {
  flex: 1 1 280px; /* cresce igual, base 280px */
  max-width: 320px;
}

/* Item que nunca encolhe (logo, ícone) */
.logo {
  flex-shrink: 0;
}
```

- ❑ A propriedade abreviada `flex: 1` é equivalente a `flex: 1 1 0%` — um atalho muito comum na prática.

# order — Reordenando sem Mudar o HTML

A propriedade `order` permite alterar a **ordem visual** dos flex items sem modificar o HTML. Por padrão, todos os itens têm `order: 0` e aparecem na ordem do documento.

Itens com valor menor aparecem primeiro. Valores negativos são permitidos e colocam o item antes dos demais.



Use `order` com cuidado! Alterar a ordem visual sem mudar o DOM pode prejudicar a **acessibilidade**, pois leitores de tela seguem a ordem do HTML, não a visual.

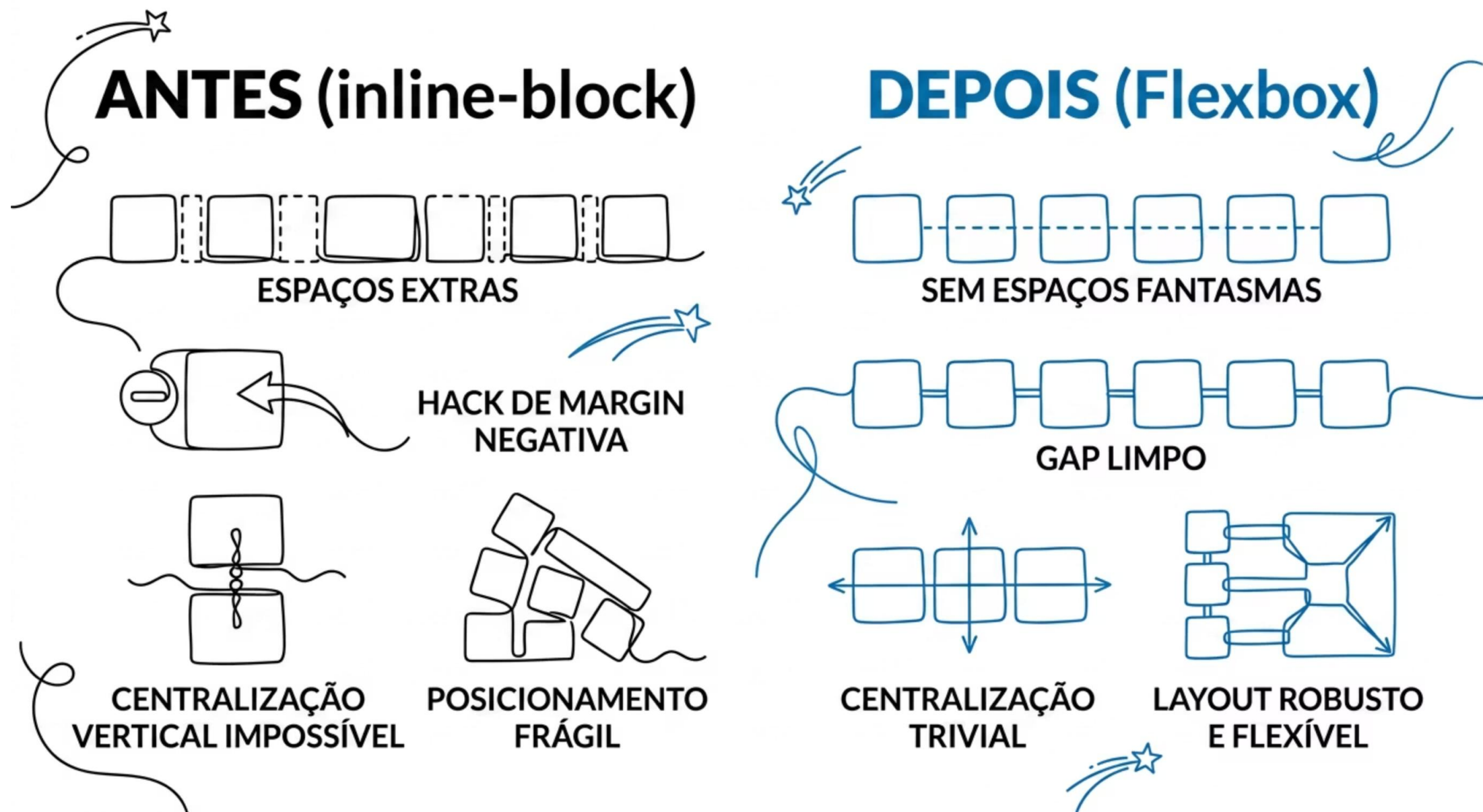
```
/* Reordenar itens da navbar em mobile
sem alterar o HTML */

/* No desktop: logo | menu | carrinho */
.navbar { display: flex; align-items: center; }

/* Em mobile, queremos: menu | logo | carrinho */
@media (max-width: 768px) {
  .navbar-logo { order: 2; } /* vai para o meio */
  .navbar-menu { order: 1; } /* vai para o início */
  .navbar-cart { order: 3; } /* fica no final */
}

/* Destaque: colocar produto em promoção
sempre como primeiro da grade */
.product-card.featured {
  order: -1; /* aparece antes de todos */
}
```

# Antes vs. Depois: O Código do E-Commerce



A diferença não é apenas estética — o código Flexbox é mais **semântico**, **previsível** e **fácil de manter**. Qualquer desenvolvedor que abrir o arquivo vai entender imediatamente a intenção do layout.

# Usando Flexbox de Forma Profissional

## 1 Use **gap** em vez de **margin**

A propriedade **gap** cria espaço entre flex items sem afetar as bordas do container. Evita o clássico problema de margem no primeiro/último elemento.

## 2 Evite larguras fixas desnecessárias

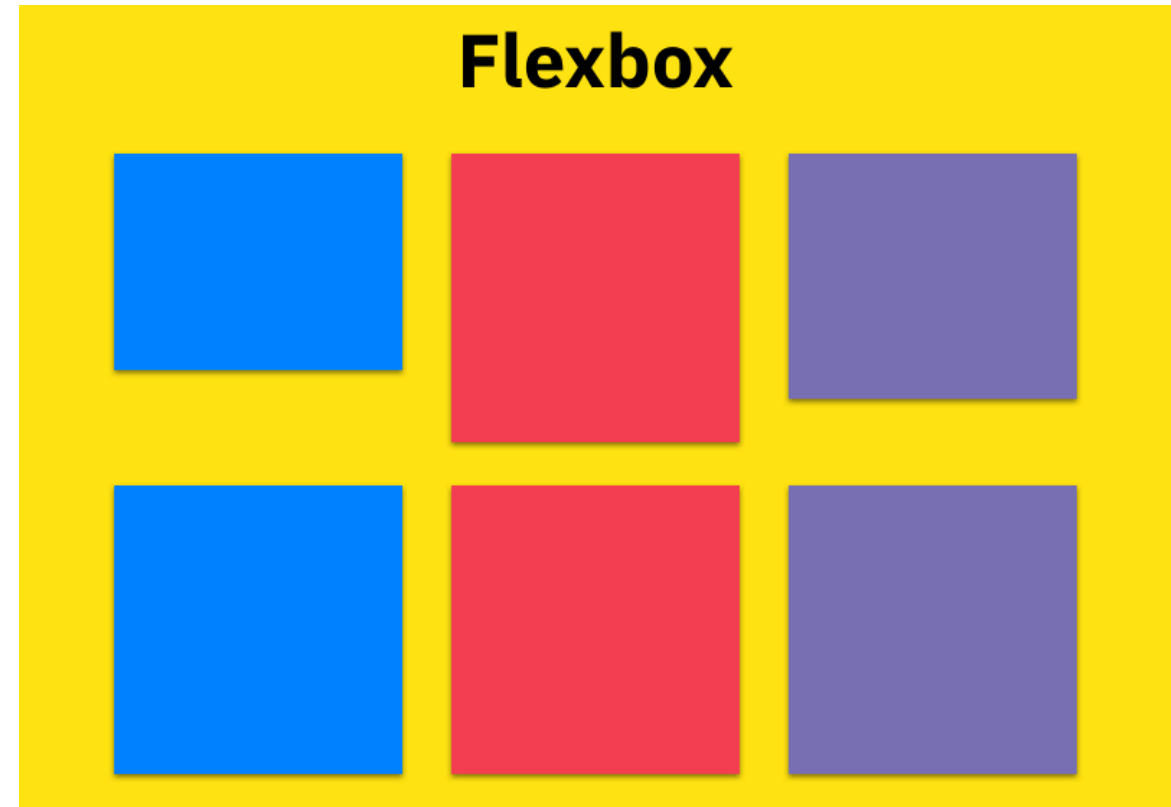
Prefira **flex-basis** e **max-width** combinados. Isso permite que o layout seja flexível dentro de limites controlados, sem engessamento.

## 3 Aninhe containers com intenção

Um flex item pode ser, ao mesmo tempo, um flex container para seus filhos. Essa composição é poderosa — use para estruturar cards, linhas e grupos com precisão.

## 4 Reserve **order** para responsividade

Alterar a ordem visual deve ter um propósito claro, geralmente dentro de media queries. Documente sempre para não confundir outros desenvolvedores da equipe.



# O Que Evitar ao Usar Flexbox

## ❌ Flexbox em Tudo

Não aplique `display: flex` em todos os elementos. Use apenas quando precisar controlar a distribuição dos **filhos diretos**.

## ❌ Ignorar o Eixo

Confundir `justify-content` com `align-items` é o erro mais comum. Lembre: o eixo muda com `flex-direction`.

## ❌ Misturar Float + Flex

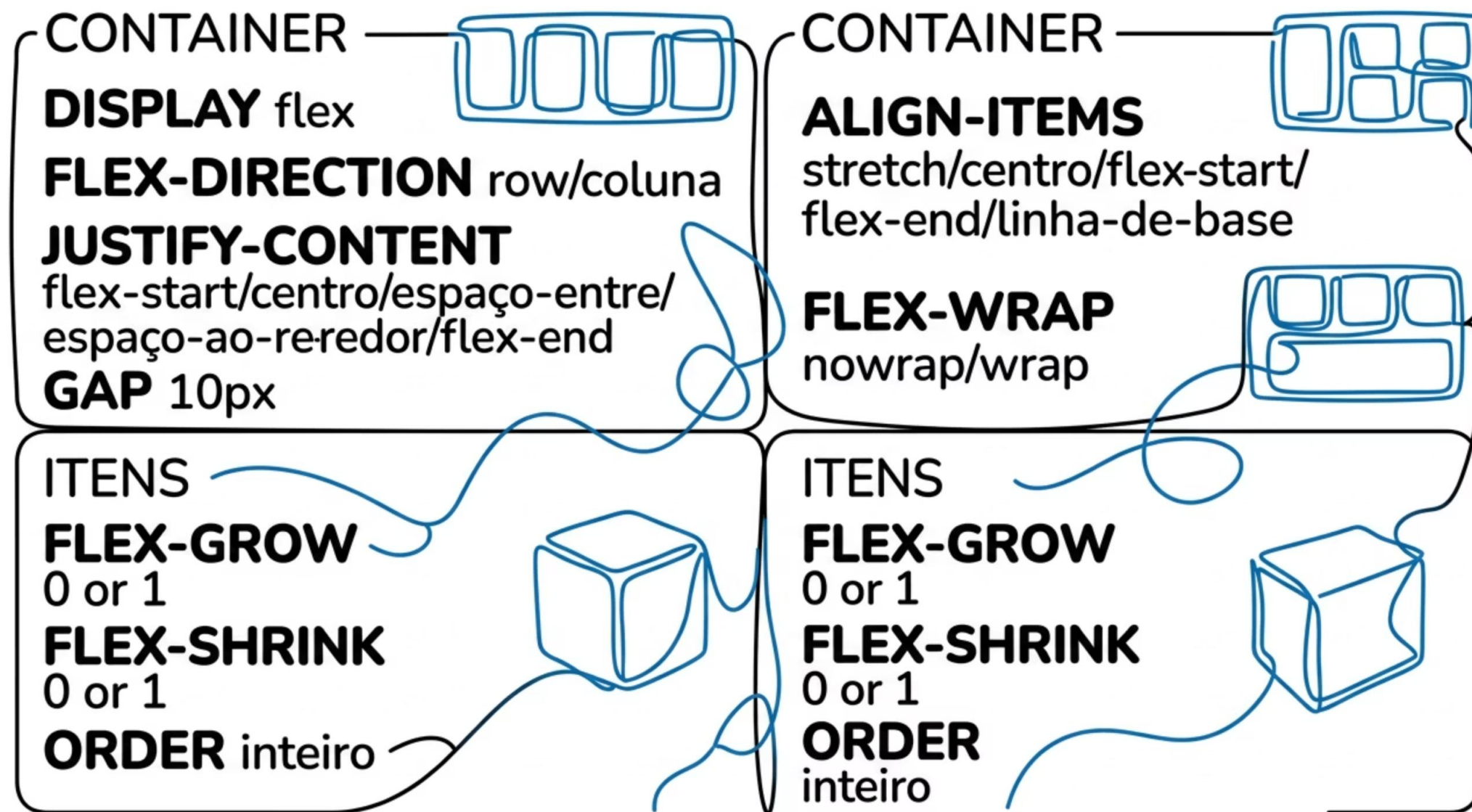
Nunca use `float` em flex items. O float é ignorado dentro de um contexto flex e só gera confusão no código.

## ❌ Esquecer `min-width: 0`

Flex items têm tamanho mínimo automático. Se o conteúdo interno estourar, adicione `min-width: 0` ao item para resolver.

💡 Dica profissional: use as **DevTools do Chrome** (aba Elements → ícone de grade) para visualizar os eixos flex e depurar layouts em tempo real.

# Resumo Visual das Propriedades



Guarde este mapa mental como referência rápida. Com o tempo, o uso de cada propriedade se tornará intuitivo — especialmente após aplicar no projeto do e-commerce.

# Conclusão: Layout Moderno com Flexbox

## ✅ O Que Aprendemos

Flex Container e Items, eixos main e cross, `justify-content`, `align-items`, `flex-wrap`, `flex-grow` e `order`.

## 🔧 O Que Aplicamos

Refatoramos navbar, hero e grade de produtos do e-commerce — removendo todos os hacks e técnicas legadas.

## 🚀 O Que Ganhamos

Código mais limpo, semântico e fácil de manter. Base sólida para responsividade e para a próxima aula de CSS Grid.

"O Flexbox não é apenas uma técnica — é uma mudança de mentalidade. Você para de lutar contra o CSS e começa a trabalhar com ele."



